FIAP

Estruturas de Decisão e Funções em C

Ciência da Computação

Prof. Dr. Érick T. Yamamoto

O que vamos aprender hoje?

FIAP

Ao final desta aula, você será capaz de:

- Dominar a estrutura de seleção múltipla **switch case**
- Entender as diferenças e quando usar **if-else vs switch**
- Aprender a modularizar código usando **funções**
- Compreender **escopo de variáveis** e passagem de parâmetros
- Praticar a integração de **switch com funções**

Foco: Código Limpo e Modularização Eficiente em C

Continuação...

FIAP

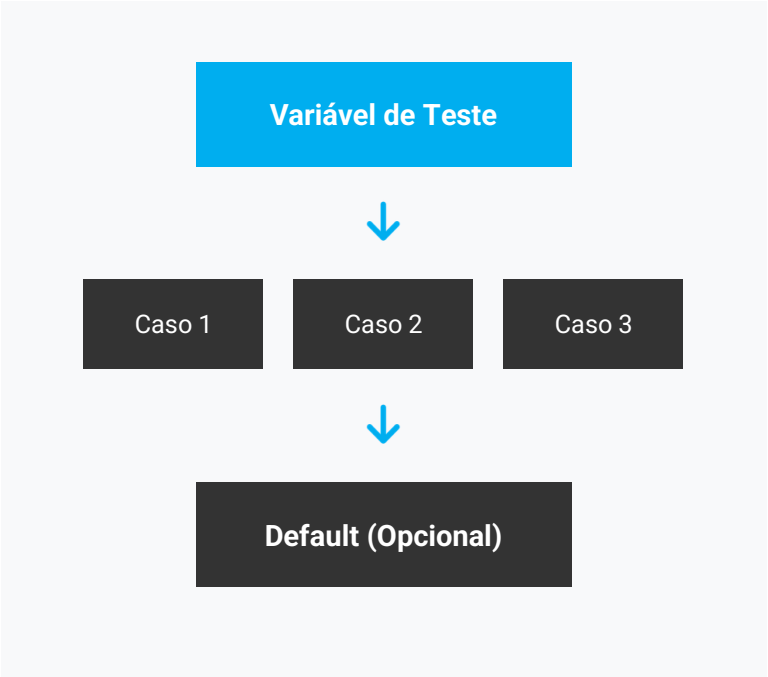
- Entender o conceito de **modularização** de código.
- Aprender a **declarar, definir e chamar** funções.
- Compreender **escopo de variáveis** e passagem por valor.
- Integrar **funções com switch case** de forma profissional.

Foco: Modularização e Reutilização de Código em C

Estrutura switch case: O que é?

FIAP

- Estrutura de controle de fluxo para testar uma variável contra uma lista de valores (casos).
- Alternativa mais limpa e organizada a múltiplos **if-else if** encadeados.
- Ideal para situações onde uma única variável é comparada com diversos valores constantes.



Sintaxe do switch case

FIAP

switch (expressao)

Avalia a variável ou expressão (deve ser do tipo integral: int ou char).

case valor:

Define um ponto de entrada se a expressão for igual ao valor constante.

break;

Interrompe a execução do bloco switch. Sem ele, o código continua nos próximos cases.

default:

Executado se nenhum dos casos anteriores for correspondido. É a "cláusula de escape".

```c
switch (opcao) {
case 1:
printf("Opcao 1");
break; // Sai do switch

case 2:
printf("Opcao 2");
break;

default:
printf("Invalido");
// break opcional aqui
}
```

Exemplo Prático 1: Menu de Opções

FIAP

```c
int opcao;
printf("1-Saldo\n2-Extrato\nEscolha: ");
scanf("%d", &opcao);
switch(opcao) {
case 1: printf("Seu saldo e R$ 100,00\n"); break;
case 2: printf("Exibindo extrato...\n"); break;
default: printf("Opcao inexistente!\n"); }
```

Análise do Código:

- O usuário insere um número inteiro que representa sua escolha no menu.
- O **switch** avalia o valor de `opcao` e salta diretamente para o **case** correspondente.
- O **break** é fundamental para que apenas um bloco seja executado.

> ***Dica:*** *O caso `default` funciona como uma rede de segurança para entradas inválidas do usuário.*

Uso de break e default

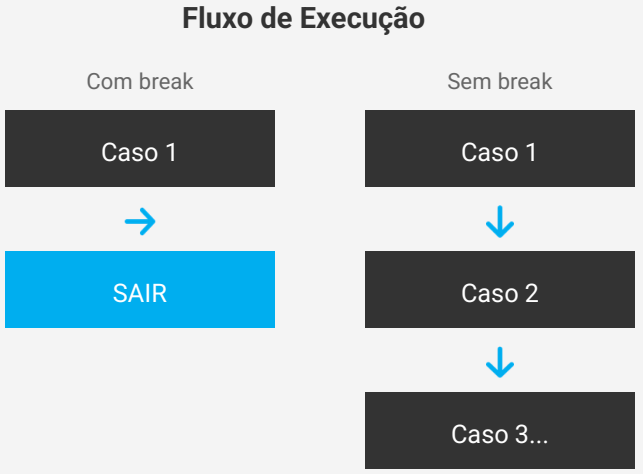
FIAP

O papel do break

- Interrompe a execução do bloco **switch** imediatamente.
- Sem ele, ocorre o efeito "**fall-through**": o código continua executando os casos seguintes.

A importância do default

- Funciona como uma cláusula de "escape" para valores não previstos.
- Garante a robustez do programa, tratando entradas inválidas ou inesperadas.



Dica: O caso `default` não precisa de `break` se for o último do bloco, mas é boa prática incluí-lo.

Comparação: if-else vs switch case

FIAP

Característica	if-else	switch case
Flexibilidade	Alta: aceita intervalos (x > 10) e múltiplas variáveis.	Baixa: apenas igualdade de uma única variável.
Tipos de Dados	Qualquer tipo (float, double, strings, etc).	Apenas inteiros (int) e caracteres (char).
Legibilidade	Pode ficar confuso com muitos níveis encadeados.	Muito organizado para menus e máquinas de estado.
Performance	Avaliação sequencial (lento para muitos casos).	Otimizado via "Jump Tables" pelo compilador.

Dica: Use **switch** para clareza em menus e **if-else** para lógicas complexas ou faixas de valores.

Boas Práticas: switch case

FIAP

- **Sempre inclua o default:** Garante que o programa trate entradas inesperadas e evita comportamentos indefinidos.
- **Use comentários para *fall-through*:** Se a ausência de um `break` for intencional, documente isso claramente para outros desenvolvedores.
- **Mantenha os cases curtos:** Se a lógica de um caso for complexa, encapsule-a em uma função para manter o switch legível.
- **Ordenação Lógica:** Organize os casos de forma que faça sentido para o domínio do problema (ex: ordem numérica ou alfabética).



"Código limpo não é apenas sobre funcionar, é sobre ser compreensível por seres humanos."

Praticar em Sala: Calculadora de Dias

FIAP

Desenvolva um programa que:

- Receba um número de 1 a 7 do usuário.
- Use **switch case** para imprimir o nome do dia da semana (1-Domingo, 2-Segunda, etc).
- Exiba "Dia inválido" para qualquer outro número usando o caso **default**.
- Não esqueça do **break** em cada caso!

Objetivo: Consolidar o uso da sintaxe básica do switch case e o tratamento de erros em sala de aula.

Por que usar funções?

FIAP

- **Modularização:** Divisão de problemas complexos em partes menores e gerenciáveis.
- **Reutilização:** Escreva uma vez, use várias vezes. Evite a duplicação de código.
- **Manutenção:** Facilidade para encontrar e corrigir erros em blocos específicos.
- **Legibilidade:** O código fica mais limpo, organizado e fácil de entender.



O Ciclo de Vida da Função

FIAP

1. Protótipo

Declaração no topo do código. Informa ao compilador que a função existe, seu nome, retorno e parâmetros.

2. Chamada

Ocorre dentro da `main` ou de outra função. É o momento em que o código da função é executado.

3. Definição

O corpo real da função. Contém a lógica de processamento e o comando de retorno.

EXEMPLO COMPLETO:

```
int somar(int a, int b); // 1. Protótipo
 int main() {
int res = somar(10, 5); // 2. Chamada
printf("Soma: %d", res);
return 0;
}
 int somar(int a, int b) { // 3. Definição
return a + b;
}
```

Comunicação entre Funções

FIAP

Tipo de Retorno

- Define o tipo de dado que a função devolve (ex: **int, float, char**).
- Use **void** quando a função apenas executa uma tarefa sem retornar valor.
- O comando **return** encerra a função e envia o dado ao chamador.

Parâmetros

- Variáveis que recebem os valores passados durante a chamada da função.
- Permitem que a função processe diferentes dados em cada execução.
- Devem ter tipo e nome definidos na assinatura da função.

Exemplos de Assinaturas

Assinatura da Função	Explicação
`int somar(int a, int b)`	Recebe dois inteiros e retorna um inteiro.
`void imprimirAviso()`	Não recebe parâmetros e não retorna nada.
`float calcularArea(float raio)`	Recebe um float e retorna um float.
`char obterConceito(float nota)`	Recebe uma nota e retorna um caractere (A, B, C...).

Escopo e Segurança de Dados

FIAP

Escopo Local vs Global

- **Local:** Declarada dentro de uma função. Só existe ali.
- **Global:** Declarada fora de funções. Acessível por todos.
- **Regra:** Prefira sempre variáveis locais para evitar erros.

Passagem por Valor

- O C passa uma **cópia** do valor para a função.
- Alterações na função **não afetam** a variável original.
- Garante a integridade dos dados fora da função.

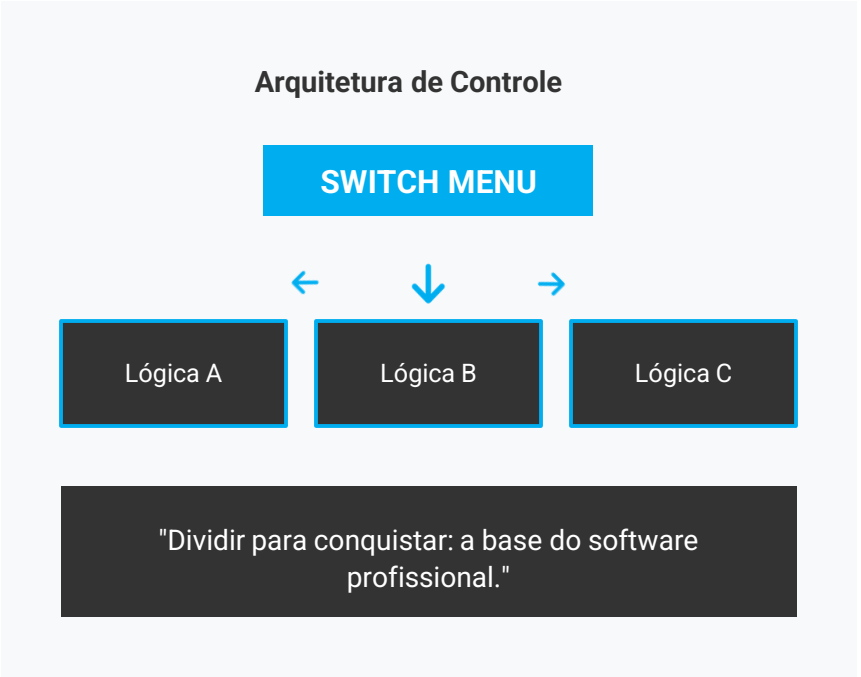
```c
void teste(int x) { x = 10; } // Altera apenas a copia local
 int main() {
int n = 5;
teste(n);
printf("%d", n); // Ainda imprime 5!
return 0;
}
```

> ***Conceito Chave:*** *Funções em C são isoladas. Elas se comunicam apenas via parâmetros e retorno.*

Padrão de Projeto: switch + funções

FIAP

- O **switch** atua como um "despachante", direcionando o fluxo para a função correta.
- Mantém a função `main()` limpa, delegando a lógica complexa para funções específicas.
- Facilita o **teste unitário**: cada função pode ser testada de forma independente.
- Melhora a colaboração: diferentes programadores podem trabalhar em funções diferentes.



Exemplo: Menu com Funções

FIAP

```c
void depositar() { printf("Deposito realizado\n"); }
void sacar() { printf("Saque realizado\n"); }
 int main() {
int opcao;
printf("1-Deposito\n2-Saque\nEscolha: ");
scanf("%d", &opcao);
 switch(opcao) {
case 1:
depositar(); // Chamada da funcao
break;
case 2:
sacar(); // Chamada da funcao
break;
default:
printf("Opcao invalida\n");
}
return 0;
}
```

Por que usar este padrão?

- **Abstração:** O `main` foca no fluxo do programa, não nos detalhes de implementação.
- **Limpeza:** O bloco `switch` fica curto e fácil de ler, servindo apenas como controle.
- **Manutenção:** Alterar a lógica de uma operação exige mexer apenas na sua função específica.

Este padrão é a base para sistemas interativos profissionais em C.

Dividir para Conquistar

FIAP

- **Simplificação:** Grandes programas são impossíveis de manter em um único bloco.
- **Módulos:** Funções permitem quebrar o problema em partes independentes.
- **Testabilidade:** Facilita o teste e a correção de bugs em blocos isolados.
- **Agilidade:** Diferentes programadores podem trabalhar em funções diferentes.



Código: Criando sua primeira função

FIAP

```c
// 1. Prototipo da Funcao
int calcularQuadrado(int n);
 int main() {
int num = 5;
int resultado;
 // 2. Chamada da Funcao
resultado = calcularQuadrado(num);
 printf("O quadrado de %d e %d\n", num, resultado);
return 0;
}
 // 3. Definicao da Funcao
int calcularQuadrado(int n) {
return n * n;
}
```

Entenda o Processo:

Protótipo: Aviso ao compilador no topo do código sobre a existência da função.

Chamada: Execução dentro da `main` passando o valor `num` como argumento.

Definição: Onde a lógica real de cálculo (`n * n`) é implementada.

> ***Dica:*** *Uma função bem escrita faz apenas **uma** coisa e a faz bem.*

Segurança: Passagem por Valor

FIAP

- Em C, a função recebe uma **cópia** da variável passada como argumento.
- Alterar o parâmetro dentro da função **NÃO altera** a variável original na main.
- Isso garante a integridade dos dados e o isolamento total das funções.

> ***Atenção:*** *A variável na main e o parâmetro na função ocupam endereços diferentes na memória.*

```c
void teste(int x) {
x = 10; // Altera apenas a copia local
}
 int main() {
int n = 5;
teste(n);
printf("%d", n); // Ainda imprime 5!
return 0;
}
```

Atividade para Casa: Mini-Sistema

FIAP

Desafio: Conversor de Temperaturas

- Crie um menu com `switch case` para escolher entre Celsius para Fahrenheit ou vice-versa.
- Implemente cada cálculo de conversão em uma **função dedicada**.
- Trate opções inválidas no menu usando o caso `default` .
- Comente seu código explicando a lógica de cada função e do switch.

Entrega: Suba o código no seu repositório do **GitHub** e envie o link via portal até a próxima aula.